

```
#include "I2Cdev.h"
```

```
#include <PID_v1.h>
```

```
#include "MPU6050_6Axis_MotionApps20.h"
```

```
MPU6050 mpu;
```

```
bool dmpReady = false;
```

```
uint8_t mpuintStatus;
```

```
uint8_t devStatus;
```

```
uint16_t packetSize;
```

```
uint16_t fifoCount;
```

```
uint8_t fifoBuffer[64];
```

```
Quaternion q;
```

```
VectorFloat gravity;
```

```
float ypr[3];
```

```
double setpoint = 182;
```

```
double Kp = 15;
```

```
double Kd = 0.9;
```

```
double Ki = 140;
```

```
double input, output;
```

```
PID pid(&input, &output, &setpoint, Kp, Ki, Kd, DIRECT);
```

```
volatile bool mpuInterrupt = false;
```

```
void dmpDataReady() {  
    mpulInterrupt = true;  
}  
  
void setup() {  
    Serial.begin(115200);  
  
    mpu.initialize();  
    mpu.testConnection();  
  
    devStatus = mpu.dmpInitialize();  
  
    mpu.setXGyroOffset(-479);  
    mpu.setYGyroOffset(84);  
    mpu.setZGyroOffset(15);  
    mpu.setZAccelOffset(1638);  
  
    if (devStatus == 0) {  
        mpu.setDMPEnabled(true);  
        attachInterrupt(0, dmpDataReady, RISING);  
        mpulntStatus = mpu.getIntStatus();  
        dmpReady = true;  
        packetSize = mpu.dmpGetFIFOPacketSize();  
  
        pid.SetMode(AUTOMATIC);  
        pid.SetSampleTime(10);  
        pid.SetOutputLimits(-255, 255);
```

```
}
```

```
pinMode(6, OUTPUT);
```

```
pinMode(9, OUTPUT);
```

```
pinMode(10, OUTPUT);
```

```
pinMode(11, OUTPUT);
```

```
analogWrite(6, LOW);
```

```
analogWrite(9, LOW);
```

```
analogWrite(10, LOW);
```

```
analogWrite(11, LOW);
```

```
}
```

```
void loop() {
```

```
    if (!dmpReady) return;
```

```
    while (!ImpulInterrupt && fifoCount < packetSize) {
```

```
        pid.Compute();
```

```
        Serial.print(input);
```

```
        Serial.print(" =>");
```

```
        Serial.println(output);
```

```
        if (input > 150 && input < 200) {
```

```
            if (output > 0)
```

```
                Forward();
```

```
            else if (output < 0)
```

```

        Reverse();
    } else {
        Stop();
    }
}

mpuInterrupt = false;
mpuIntStatus = mpu.getIntStatus();

fifoCount = mpu.getFIFOCount();

if ((mpuIntStatus & 0x10) || fifoCount == 1024) {
    mpu.resetFIFO();
    Serial.println("FIFO overflow!");
} else if (mpuIntStatus & 0x02) {
    while (fifoCount < packetSize) fifoCount = mpu.getFIFOCount();

    mpu.getFIFOBytes(fifoBuffer, packetSize);
    fifoCount -= packetSize;

    mpu.dmpGetQuaternion(&q, fifoBuffer);
    mpu.dmpGetGravity(&gravity, &q);
    mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);

    input = ypr[1] * 180 / M_PI + 180;
}
}

```

```
void Forward() {  
    analogWrite(6, output);  
    analogWrite(9, 0);  
    analogWrite(10, output);  
    analogWrite(11, 0);  
    Serial.print("F");  
}
```

```
void Reverse() {  
    analogWrite(6, 0);  
    analogWrite(9, output * -1);  
    analogWrite(10, 0);  
    analogWrite(11, output * -1);  
    Serial.print("R");  
}
```

```
void Stop() {  
    analogWrite(6, 0);  
    analogWrite(9, 0);  
    analogWrite(10, 0);  
    analogWrite(11, 0);  
    Serial.print("S");  
}
```